

Loan Repayment Risk Prediction Using an Imbalance-Aware and Interpretable Machine Learning Pipeline

Rohith G S
Dept. of CSE (AI&ML)
PES University

Nakshathira B
Dept. of CSE (AI&ML)
PES University

Abstract—This paper presents an applicant-level machine learning pipeline for predicting loan repayment difficulty from the Home Credit multi-table dataset. Seven related tables are validated and reduced to one row per applicant through historical aggregation, compact financial and repayment features, and leakage-safe preprocessing. Logistic regression, random forest, and XGBoost are compared under a fixed random seed and stratified train/validation/test protocol. A controlled 20-trial XGBoost experiment is selected using validation PR-AUC only, with ROC-AUC and F1 as tie-breakers. The accepted tuned XGBoost model uses a validation-selected decision threshold of 0.58 and achieves a final untouched-test ROC-AUC of 0.7781, PR-AUC of 0.2738, precision of 0.2561, recall of 0.4729, and F1-score of 0.3322. The resulting artifacts support feature-importance analysis and a lightweight demonstration application.

Index Terms—credit risk, loan repayment, class imbalance, XGBoost, tabular machine learning, threshold optimization

I. INTRODUCTION

Loan repayment difficulty is an imbalanced binary classification problem: the positive class is substantially smaller than the non-default class, while the available information is distributed across current application, credit-history, and repayment tables. The objective of this project is to produce a reproducible applicant-level risk score rather than a row-level transaction classifier. The design therefore prioritizes leakage control, compact domain-driven features, imbalance-aware evaluation, and an interpretable delivery path.

II. DATASET AND AGGREGATION

The input consists of the seven local Home Credit CSV tables shown in Table I. Required columns and key relationships were validated before feature construction. Historical records were joined through `SK_ID_CURR`; bureau history was connected through `SK_ID_BUREAU`, and prior-loan history through `SK_ID_PREV`. Each historical table was reduced to applicant-level aggregates so that the final modeling table contains one row per `SK_ID_CURR`. Large tables were processed in a memory-conscious manner where practical, and raw CSV files remain local rather than being committed.

III. METHODOLOGY

A. Feature Engineering

Current-application variables were combined with valid financial ratios such as credit-to-income and annuity-to-income.

TABLE I
VALIDATED SOURCE TABLES

Table	Rows	Columns
application_train	307,511	122
bureau	1,716,428	17
bureau_balance	27,299,925	3
previous_application	1,670,214	37
POS_CASH_balance	10,001,358	8
installments_payments	13,605,401	8
credit_card_balance	3,840,312	23

Bureau features summarize account counts, active and overdue behavior, credit and debt amounts, and bureau-balance history. Previous-application features include application counts, approval/refusal behavior, and historical amounts. POS/CASH, installments, and credit-card tables contribute history counts, delinquency statistics, payment delays, payment-to-installment ratios, balance/limit information, and related repayment summaries. `TARGET` was used only as the response and never to construct predictors.

B. Splitting and Preprocessing

The data were split into stratified training, validation, and test partitions using random seed 42. Learned preprocessing was fitted on the training partition only and then applied to validation and test data. Numeric missing values were imputed and categorical variables were encoded through the saved preprocessing bundle; scaling was applied where required by the model. The validation partition was reserved for model comparison, tuning, and threshold selection. The test partition was not used until the selected model and threshold had been frozen.

C. Models and Imbalance Handling

The baseline comparison included logistic regression, random forest, and XGBoost. Class weighting was preferred to synthetic oversampling, using the appropriate positive-class weighting parameter for boosting. XGBoost was the strongest baseline candidate and was therefore the only model tuned. The fallback design permits histogram gradient boosting if XGBoost is unavailable, but the reported accepted model is XGBoost.

TABLE II
BASELINE VALIDATION COMPARISON

Model	ROC-AUC	PR-AUC	Precision	Recall
Logistic regression	0.7599	0.2366	0.1671	0.6910
Random forest	0.7409	0.2076	0.2250	0.3635
XGBoost	0.7722	0.2591	0.1885	0.6405

TABLE III
FINAL TUNED XGBOOST TEST EVALUATION

Metric	Value
Decision threshold	0.58
ROC-AUC	0.7781
PR-AUC	0.2738
Precision	0.2561
Recall	0.4729
F1-score	0.3322
True negatives	49,716
False positives	6,822
False negatives	2,617
True positives	2,348

D. Controlled Tuning and Thresholding

Twenty deterministic XGBoost configurations were sampled with seed 42. The search varied tree count and depth, learning rate, row and column subsampling, child-weight, regularization, and `scale_pos_weight`. Validation PR-AUC was the primary selection metric, with validation ROC-AUC and F1 as tie-breakers. The tuned validation PR-AUC increased from 0.2591 to 0.2616 and validation ROC-AUC increased from 0.7722 to 0.7745. After accepting the tuned candidate, the classification threshold was selected on validation data by maximizing F1. The selected threshold was 0.58; its validation precision, recall, and F1 were 0.2511, 0.4757, and 0.3287, respectively.

IV. EXPERIMENTAL RESULTS

Table II reports the baseline comparison at threshold 0.50. XGBoost provided the highest baseline ROC-AUC and PR-AUC, motivating the controlled tuning stage.

The final accepted model was evaluated once on the untouched test partition after threshold selection. The test set contains 61,503 rows and has a positive rate of 0.0807. As shown in Table III, the model identifies 2,348 true positives and 2,617 false negatives at threshold 0.58. The relatively high recall is appropriate for a screening-oriented risk application, while precision reflects the cost of investigating a minority-risk population.

Permutation importance was generated from validation data, and the demonstration application exposes the frozen model probability, threshold decision, derived financial indicators, individual prediction contributions, and overall model signals. The application does not train, tune, or evaluate the test set.

V. LIMITATIONS

The experiment uses a compact, manually defined feature set and a limited 20-trial search rather than exhaustive optimization. Results depend on the fixed split, seed, local data quality, and the selected operating objective of validation F1. A single threshold cannot represent every operational cost trade-off, and the reported precision and recall should not be interpreted as calibrated business probabilities without further calibration and monitoring. Historical aggregates may also encode population or time effects that require external validation before deployment. Finally, the demonstration is intentionally lightweight and is not a production lending decision system.

VI. CONCLUSION

This project demonstrates a complete leakage-safe workflow for applicant-level loan repayment risk prediction. Seven tables were validated and aggregated, compact financial and repayment features were created, and three baseline models were compared under a reproducible imbalance-aware protocol. A controlled XGBoost experiment produced a modest validation improvement and was accepted using validation-only selection. The final tuned model, with a threshold of 0.58 selected before test evaluation, achieved ROC-AUC 0.7781, PR-AUC 0.2738, precision 0.2561, recall 0.4729, and F1-score 0.3322 on the untouched test set. These results provide a consistent basis for the accompanying report, presentation, explainability outputs, and demo.

REFERENCES

- [1] Home Credit Group, "Home Credit Default Risk," Kaggle competition dataset, 2018. [Online]. Available: <https://www.kaggle.com/competitions/home-credit-default-risk>
- [2] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [3] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [4] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.